Lezione 13 - Applicazione in Python: programmazione lineare e ALM deterministico

Metodi Quantitativi per la Finanza

Paolo Falbo, Cristian Pelizzari

Dipartimento di Economia e Management



Università di Brescia

Dalla teoria alla soluzione computazionale

La Lezione 13 consolida operativamente i modelli introdotti nelle due lezioni precedenti.

$$\text{Lezione 11} \longrightarrow \text{Lezione 12} \longrightarrow \text{Lezione 13}$$

progr. lin. $\rightarrow$ ALM deterministico $\rightarrow$ implementazione e analisi computaz.

- La lezione 11 ha introdotto la scelta ottima sotto vincoli, la dualità e i valori marginali.
- La lezione 12 ha introdotto la dimensione temporale dei cash flow, i bilanci di liquidità e il funding.
- In questa lezione, il modello viene tradotto nella forma richiesta dal solver, risolto, controllato e sottoposto ad analisi what-if.

Idea guida: non introduciamo una nuova classe di modelli; rendiamo operativo il modello già costruito.

Obiettivi e struttura della lezione

La lezione persegue tre obiettivi:

1. **Un piccolo approfondimento Python**
   Isolare una struttura del codice utile per costruire e analizzare ripetutamente un modello parametrico.
2. **Il Caso Aula**
   Tradurre un problema di ALM deterministico nella forma richiesta dal solver, verificare la soluzione e impostare benchmark, sensitività, frontiera di fattibilità e scenari.
3. **Il Caso TakeHome**
   Presentare un nuovo problema finanziario al quale trasferire autonomamente lo stesso metodo di lavoro, senza svilupparne la soluzione in aula.

La lezione sarà articolata in due momenti:

inquadramento con le slide $\longrightarrow$ sviluppo del Caso Aula in VS Code.

Priorità: il Caso Aula costituisce il nucleo principale della lezione; le slide ne preparano lo sviluppo computazionale senza sostituirlo.

Dal codice ripetuto a una funzione

Nel Caso Aula, lo stesso modello deve essere risolto più volte al variare di alcuni parametri.

Una possibilità sarebbe ripetere ogni volta le stesse istruzioni. La soluzione più naturale è invece racchiudere il procedimento in una **funzione**.

$$(\theta, \phi, \psi) \quad \longrightarrow \quad \text{costruzione del problema} \quad \longrightarrow \quad \text{soluzione.}$$

In Python, la struttura essenziale è:

```
def solve_model(theta, phi, psi):
    ...
    return result
```

Idea chiave: la funzione rende il modello ripetibile senza duplicare la logica di costruzione e soluzione.

Input, elaborazione e output

Una funzione può essere letta come una trasformazione organizzata tra **input** e **output**.

$$\boxed{\theta, \phi, \psi} \longrightarrow \boxed{\text{elaborazione}} \longrightarrow \boxed{\text{result}}$$

Nel nostro caso, l'elaborazione comprende:

1. costruire i coefficienti che dipendono dai parametri;
2. costruire uguaglianze, disuguaglianze e bound;
3. chiamare il solver;
4. restituire il risultato della soluzione.

L'oggetto restituito dal solver contiene poi le informazioni necessarie per analizzare:

soluzione , valore obiettivo , fattibilità , slack e marginali.

Collegamento metodologico: la struttura della funzione riflette la scomposizione del problema in tappe input–output.

Da una soluzione a una famiglia di soluzioni

La funzione parametrica permette di utilizzare lo stesso modello per domande quantitative diverse.

1. **Benchmark**

   ```
   solve_model(1, 1, 1)
   ```

   fornisce la soluzione del caso di riferimento.
2. **Sensitività univariata**

   ```
   solve_model(theta, 1, 1)
   ```

   permette di studiare come cambia la soluzione al variare di un solo driver.
3. **Scenari congiunti**

   ```
   solve_model(theta, phi, psi)
   ```

   permette di modificare simultaneamente più driver.

una funzione $\rightarrow$ una famiglia di problemi LP $\rightarrow$ una famiglia di soluzioni

Passaggio successivo: identifichiamo ora il modello ALM che la funzione dovrà costruire e risolvere.

Caso Aula: Silicon Valley Bank 2023

Consideriamo un modello stilizzato di Asset–Liability Management ispirato al caso Silicon Valley Bank.
La banca deve scegliere un piano di allocazione degli attivi e di gestione della liquidità su quattro date, potendo ricorrere anche a funding esterno.
La domanda quantitativa centrale è:

come cambia il piano ALM ottimo quando cambiano fabbisogni di liquidità, capacità di funding e costo del funding?

I tre driver dell'analisi sono:

$$\begin{aligned} \theta &: \text{fabbisogni di liquidità,} \\ \phi &: \text{capacità di funding,} \\ \psi &: \text{costo del funding.} \end{aligned}$$

Una seconda domanda riguarda la **fattibilità**: fino a quale livello possono aumentare i fabbisogni iniziali prima che non esista più un piano ammissibile?

Dalla Lezione 11 alla Lezione 12

Il Caso Aula combina due livelli già sviluppati nel corso.

Lezione 11

$$x = (x_1, \ldots, x_4)'$$

- allocazione degli attivi;
- funzione obiettivo;
- vincoli;
- soluzione ottima;
- dualità e valori marginali.

Lezione 12

$$(x, g, u)$$

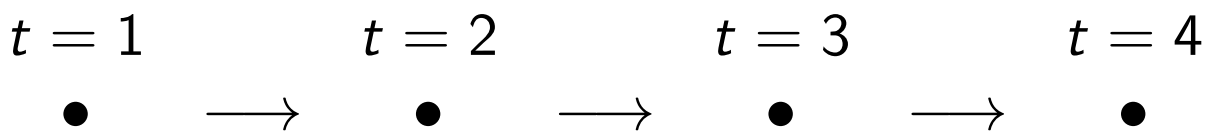
- cash flow distribuiti nel tempo;
- bilanci di liquidità;
- liquidità riportata;
- funding esterno;
- valori marginali temporali.

La classe matematica resta la programmazione lineare

ma la struttura economica diventa multiperiodale.

Il tempo entra nel modello

Nel modello ALM, la liquidità deve essere disponibile alla data in cui nasce il fabbisogno.



A ogni data, intervengono:

- i cash flow generati dagli attivi, $a_t' x$;
- il fabbisogno di liquidità, d_t;
- l'eventuale funding esterno, u_t;
- la liquidità residua trasferita alla data successiva, g_t.

dal "quanto" al "quanto e quando"

Il problema non riguarda quindi soltanto **quanto** cash flow viene generato, ma anche **quando** viene generato.

Le variabili decisionali

Il piano ALM comprende tre gruppi di variabili decisionali.

Allocazione degli attivi

$$x = (x_1, x_2, x_3, x_4)'.$$

x_j indica l'ammontare investito nella classe di attivo j.

Liquidità riportata

$$g = (g_1, g_2, g_3, g_4)'.$$

g_t rappresenta la liquidità residua dopo il fabbisogno alla data t.

Funding esterno

$$u = (u_1, u_2, u_3)'.$$

u_t rappresenta il funding ottenuto alla data t, disponibile immediatamente e rimborsato alla data successiva.

Il modello contiene complessivamente $4 + 4 + 3 = \boxed{11}$ variabili decisionali.

La funzione obiettivo

Il piano viene scelto massimizzando il valore economico complessivo:

$$\max_{x,g,u} \Pi(x, g, u; \psi) = c'x + \sum_{t=1}^{3} r_t^g g_t - \sum_{t=1}^{3} \psi r_t^u u_t.$$

La funzione obiettivo combina tre componenti:

1. $c'x$: rendimento economico dell'allocazione degli attivi;
2. $\sum_{t=1}^{3} r_t^g g_t$: remunerazione della liquidità riportata;
3. $-\sum_{t=1}^{3} \psi r_t^u u_t$: costo del funding esterno.

$$\boxed{\psi}$$

modifica quindi direttamente il costo economico del funding.

Bilanci di liquidità: la struttura ricorsiva

Alla prima data:

$$a_1' x + u_1 - g_1 = d_1(\theta).$$

Alle date successive, compare una struttura ricorsiva:

$$a_t' x + (1 + r_{t-1}^g) g_{t-1} + u_t - (1 + \psi r_{t-1}^u) u_{t-1} - g_t = d_t(\theta), \qquad t = 2, 3.$$

La logica è sempre la stessa:

$$\text{cash flow} + \text{liquidità riportata} + \text{nuovo funding}$$

$$=$$

$$\text{fabbisogno} + \text{rimborso} + \text{liquidità residua}$$

Punto chiave: il funding ottenuto alla data t migliora la liquidità corrente, ma genera un rimborso alla data successiva.

Gli altri vincoli

Ai bilanci temporali, si aggiungono gli altri vincoli del piano ALM.

1. **Risorse iniziali**
$$\sum_{j=1}^{4} x_j = A_0 = 100.$$
2. **Limite prudenziale**
$$\ell' x \leq K, \qquad K = 7.$$
3. **Concentrazione**
$$x_3 \leq 30.$$
4. **Capacità di funding**
$$0 \leq u_t \leq \phi \bar{u}_t.$$
5. **Non negatività**
$$x_j \geq 0, \qquad g_t \geq 0.$$

Tutte le relazioni sono lineari: il problema resta quindi un problema di **programmazione lineare**.

Il vettore del solver

Per costruire il problema numerico, dobbiamo raccogliere tutte le variabili decisionali in un unico vettore.

Indichiamo localmente con y il vettore utilizzato nella forma solver:

$$y = (x_1, x_2, x_3, x_4, g_1, g_2, g_3, g_4, u_1, u_2, u_3)'.$$

Il significato economico delle variabili non cambia, ma il solver le tratta come componenti di un unico vettore numerico.

modello economico $\longrightarrow$ vettore decisionale numerico

Dalla formulazione agli input del solver

Il solver richiede quattro oggetti fondamentali.

funzione obiettivo	$\longrightarrow$	`c`
uguaglianze	$\longrightarrow$	`A_eq, b_eq`
disuguaglianze	$\longrightarrow$	`A_ub, b_ub`
limiti sulle variabili	$\longrightarrow$	`bound`

La forma risolta da `linprog` è:

$$\min_y f'y$$

soggetto a

$$A_{ub}y \leq b_{ub}, \qquad A_{eq}y = b_{eq}.$$

Idea chiave: la programmazione consiste nel tradurre correttamente gli oggetti del modello in questi input, non nel modificare il modello.

Una sola inversione di segno

Il modello economico massimizza:

$$\max_y p'y.$$

Poiché `linprog` minimizza, definiamo:

$$f = -p.$$

Il solver risolve quindi:

$$\min_y f'y.$$

Una volta ottenuta la soluzione y^*:

$$\Pi^* = p'y^* = -f'y^*.$$

il cambio di segno si effettua una sola volta

Tre parametri, tre punti di ingresso

I tre driver modificano parti diverse del problema.

$$\theta \longrightarrow b_{eq}$$

$$\phi \longrightarrow \texttt{bound}$$

$$\psi \longrightarrow f \text{ e } A_{eq}$$

In particolare:

$$d(\theta) = (30\theta, 30\theta, 22, 15)',$$

$$\bar{u}_t(\phi) = \phi \bar{u}_t.$$

Per ψ, occorre ricordare due effetti:

- modifica il costo del funding nella funzione obiettivo;
- modifica il rimborso del funding nei bilanci temporali.

Punto chiave: parametrizzare il modello significa sapere esattamente quale oggetto del solver deve cambiare.

Che cosa faremo in VS Code

A questo punto, il modello è pronto per essere costruito e analizzato numericamente.

benchmark $\longrightarrow$ controlli $\longrightarrow$ sensitività $\longrightarrow$ frontiera $\longrightarrow$ scenari

Nel laboratorio, dovremo:

- risolvere il caso benchmark;
- controllare fattibilità, residui, slack, bound attivi e marginali;
- variare separatamente θ, ϕ e ψ;
- individuare la frontiera di fattibilità rispetto a θ;
- confrontare scenari con variazioni congiunte dei driver.

Obiettivo finale: interpretare non soltanto il valore ottimo, ma l'intero piano ALM x^*, g^*, u^*.

Caso TakeHome: First Republic Bank 2023

Il caso TakeHome trasferisce lo stesso metodo di lavoro a un problema ALM differente, ispirato al caso First Republic Bank.
La struttura del problema presenta alcuni elementi nuovi:

- cinque classi di attivo;
- quattro date di bilancio;
- due possibili ondate di deposit run;
- capacità di funding limitata;
- vincoli di ristrutturazione del bilancio.

I tre driver dell'analisi sono $\boxed{\alpha}$ $\boxed{\beta}$ $\boxed{\phi}$, a cui corrispondono i fabbisogni:

$$d(\alpha, \beta) = (32\alpha, 18, 14, 32\beta)'.$$

Due vincoli caratterizzano in particolare il caso:

$$x_3 + x_4 \geq 35, \qquad x_5 \leq 35.$$

Idea chiave: il contesto finanziario cambia, ma la struttura metodologica della soluzione resta la stessa.

Dal Caso Aula al Caso TakeHome

Il caso TakeHome non è solo una variazione parametrica del Caso Aula.

Caso Aula	Caso TakeHome
Silicon Valley Bank	First Republic Bank
4 attivi	5 attivi
θ, ϕ, ψ	α, β, ϕ
stress sui fabbisogni iniziali	due ondate di deposit run
vincolo di concentrazione	vincolo legacy

La sequenza metodologica resta invece comune:

$$\boxed{\text{modello}} \longrightarrow \boxed{\text{forma solver}} \longrightarrow \boxed{\text{benchmark}} \longrightarrow \boxed{\text{controlli}}$$

$$\longrightarrow \boxed{\text{sensitività e scenari}} \longrightarrow \boxed{\text{interpretazione}}.$$

Obiettivo del TakeHome: verificare la capacità di trasferire autonomamente il metodo a una struttura finanziaria differente.

Dal modello al laboratorio in VS Code

Abbiamo definito il problema economico, la forma matematica e la traduzione richiesta dal solver.

Ora passiamo allo sviluppo operativo del Caso Aula in VS Code.

$$\boxed{\text{dati}} \longrightarrow \boxed{\text{funzione solver}} \longrightarrow \boxed{\text{benchmark}} \longrightarrow \boxed{\text{controlli}}$$

$$\longrightarrow \boxed{\text{sensitività}} \longrightarrow \boxed{\text{frontiera}} \longrightarrow \boxed{\text{scenari}}.$$

Durante lo sviluppo, dovremo verificare continuamente la coerenza tra:

$$\text{modello teorico} \quad \longleftrightarrow \quad \text{codice} \quad \longleftrightarrow \quad \text{output}.$$

Obiettivo: costruire una soluzione computazionale comprensibile, verificabile e interpretabile.